Travelling Salesman Problem

Greedy (Nearest Neighbor) Technique vs. Simulated Annealing

AI Assignment 4 — Report

2205100 (Maruf Hasan)

1. Introduction

The Travelling Salesman Problem (TSP) is simple to state but hard to solve: a salesman has to visit every city once and get back to where he started, using the shortest possible route. The tricky part is that the number of possible routes explodes as the number of cities grows, so for anything beyond a handful of cities, checking every route by brute force just isn't realistic.

For this assignment I implemented and compared two ways of solving it:

- Greedy (Nearest Neighbor) — a quick heuristic that always jumps to the closest city that hasn't been visited yet.
- Simulated Annealing (SA) — a slower, randomized method that starts from some tour and keeps tweaking it, sometimes even accepting a worse tour on purpose, so it doesn't get stuck in a bad solution.

Both algorithms work on any N × N cost matrix ($2 \le N \le 500$, always starting from City 0). I tested them on five instance sizes — 10, 20, 50, 100 and 200 cities — and ran the four experiments the assignment asks for: greedy vs SA, changing the starting temperature, changing the cooling rate, and comparing random vs greedy starting tours. All the numbers in this report come straight from actual program runs (see the exp1–exp4 CSV files).

2. How the Algorithms Work

2.1 Greedy (Nearest Neighbor)

This one is straightforward. Start at City 0, and at every step just jump to whichever unvisited city is cheapest to reach from where you currently are. Keep doing that until every city has been visited, then head back to City 0 to close the loop. In code, this means:

- Keep a visited[] list and a pointer to the current city, starting at City 0.
- At each step, look at every unvisited city and pick the cheapest one to move to.
- Do this $N - 1$ times, then add the final trip back to City 0.
- Add up the cost as you go.

The problem with this approach is that it never looks ahead. It just grabs whatever's cheapest right now, which can leave it stuck making an expensive final move later — for example, the very last unvisited city might be far from everything. That's the classic weakness of greedy algorithms, and it's what the experiments later in this report actually show happening.

2.2 Simulated Annealing

Simulated Annealing borrows its idea from how metal cools and hardens. Instead of always taking the best move like greedy does, it explores around randomly and gradually becomes more "picky" over time, controlled by a temperature value that keeps dropping.

2.2.1 How a Tour is Stored

A tour is just a list of city numbers, with City 0 always fixed at the front. So [0, 3, 1, 4, 2] means the route 0 → 3 → 1 → 4 → 2 → 0. Only the cities after position 0 ever get shuffled around.

2.2.2 Starting Tour

Both starting options mentioned in the assignment were built:

- Random start — shuffle cities 1 through N−1 randomly and put them after City 0.
- Greedy start — just use whatever tour the greedy algorithm already produced.

Experiment 4, later in this report, compares how much this choice actually matters.

2.2.3 Making a Neighbor: 2-opt

To get a "neighbor" tour, pick two random positions in the tour (never touching City 0) and reverse everything in between. This is the standard 2-opt move — it swaps out two connections in the tour for two different ones, which is often enough to break out of the kind of dead end that simple swaps can't fix.

2.2.4 When to Accept a Worse Move

Say Δ is how much worse the new tour is (new cost minus current cost). If Δ is zero or negative, the new tour is just as good or better, so it's always accepted. If it's worse (Δ > 0), it still gets accepted sometimes, with probability e^(−Δ/T). The program draws a random number between 0 and 1 and accepts the move if that number is smaller than this probability. Early on, when T is high, this probability is close to 1, so almost anything gets accepted. As T drops, the algorithm becomes much stricter.

2.2.5 Cooling Down

The temperature drops geometrically: T_new = α × T_current, with a fixed number of moves tried at each temperature before it cools further. The run stops once T falls below T_min or the iteration limit is hit. The best tour found is tracked separately, so even if the current tour gets worse temporarily (which is the whole point), the reported "best" never gets worse over time.

2.2.6 Default Settings

Unless a particular experiment was testing a parameter directly, these baseline values were used (matching the assignment's suggestion): T0 = 1000, α = 0.995, T_min = 0.001, 100 moves tried per temperature, a cap of 100,000 iterations, greedy starting tour, and a fixed random seed (only Experiment 4 varies the seed, across 5 different values, since that experiment needs multiple runs).

2.3 Code Structure

The program is split into the functions the assignment asks for: readInput(), calculateTourCost(), greedyTSP(), generateRandomTour(), generateNeighbor(), simulatedAnnealing(), and printResults(). There's also a validateTour() function used while testing, which just double-checks that a tour starts and ends at City 0, visits every

city exactly once, and has no repeats. Timing was measured around each algorithm's actual work only, not around reading input or printing output.

3. Test Instances

Five instances were used, matching the sizes the assignment recommends: 10, 20, 50, 100, and 200 cities. Each one was generated from random points with a fixed seed, so the same cost matrix is reused across all four experiments — that way, whatever changes in the results is actually because of the parameter being tested, not because the underlying map of cities changed. The program can handle asymmetric costs too, but the instances used here are symmetric, with Cost(i,i) = 0 and every other cost positive, as required.

4. Experimental Results

4.1 Experiment 1: Greedy vs. Simulated Annealing

For each instance size, SA was run starting from the greedy tour, using the baseline settings (T0=1000, α=0.995). Starting SA from the same tour greedy produces makes it a fair comparison — any improvement is purely down to SA's search, not a lucky starting point.

Cities (N)	Greedy Cost	SA Best Cost	Improvement (%)	Greedy Time (s)	SA Time (s)
10	3957.32	3059.29	22.69%	0.000002	0.008383
20	4646.49	3841.28	17.33%	0.000003	0.009888
50	7001.96	6279.45	10.32%	0.000010	0.016723
100	9668.24	8391.31	13.21%	0.000029	0.028288
200	13360.71	13360.71	0.00%	0.000092	0.051157

Table 1. Greedy vs. Simulated Annealing across instance sizes.

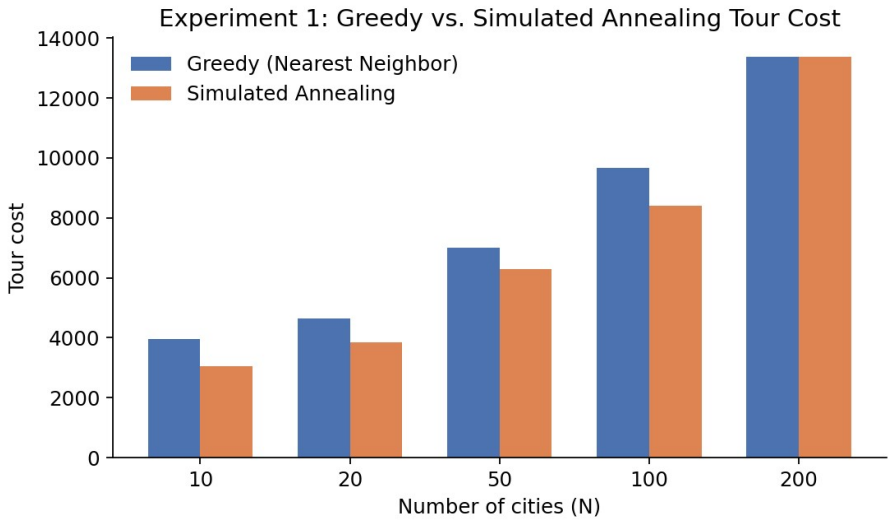


Figure 1. Tour cost comparison, Greedy vs. SA.

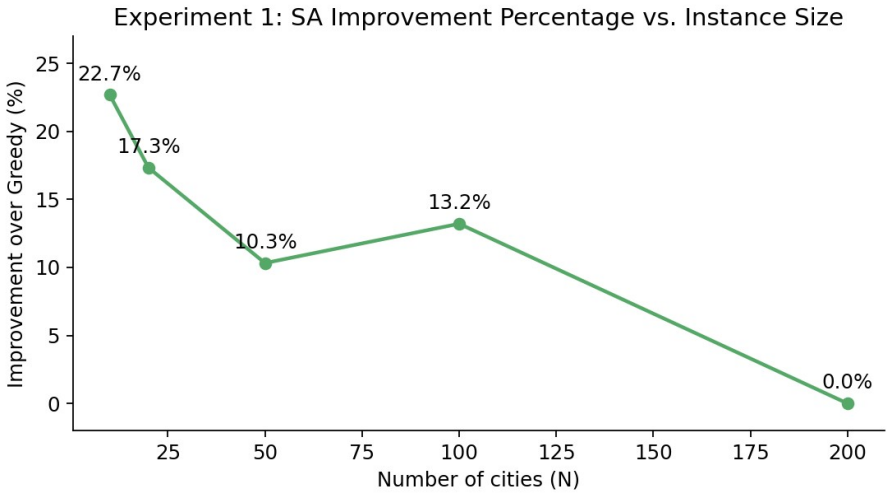


Figure 2. SA's percentage improvement over Greedy by instance size.

For instances up to 100 cities, SA beats greedy by a decent margin — anywhere from 10% to almost 23%. That's a good demonstration of what the assignment is getting at: nearest-neighbor makes a fast but shortsighted decision at every step, and SA is able to clean up a lot of that mess afterward. The biggest gain shows up at N=10 (22.7%), which makes sense since SA has plenty of iterations relative to how small the search space is. As N grows, the improvement shrinks a bit, down to 13.2% at N=100.

At N=200 though, SA didn't improve on greedy at all — it landed on exactly the same cost. This isn't SA failing as an idea, it's more a sign that 100,000 iterations just isn't enough once the search space gets much bigger. With more cities, there are far more possible 2-opt swaps to try, so the same iteration budget covers a much smaller fraction of the space. It's a useful reminder that SA's settings need to scale with problem size — a fixed iteration count that works fine for 50 cities won't necessarily cut it for 200. As for time, SA is naturally much slower than greedy (greedy finishes almost instantly), which is just the cost of doing all that extra searching.

4.2 Experiment 2: Different Initial Temperatures

Keeping N=50 and everything else at the baseline (α=0.995, greedy start), T0 was tested at three values: 100, 1000, and 5000.

T0	Best SA Cost	Accepted Moves	Worse Accepted	SA Time (s)
100	6243.13	1599	789	0.016614
1000	6279.45	24466	12254	0.016881
5000	6348.17	53683	26794	0.017478

Table 2. Effect of initial temperature T0 on Simulated Annealing (N=50).

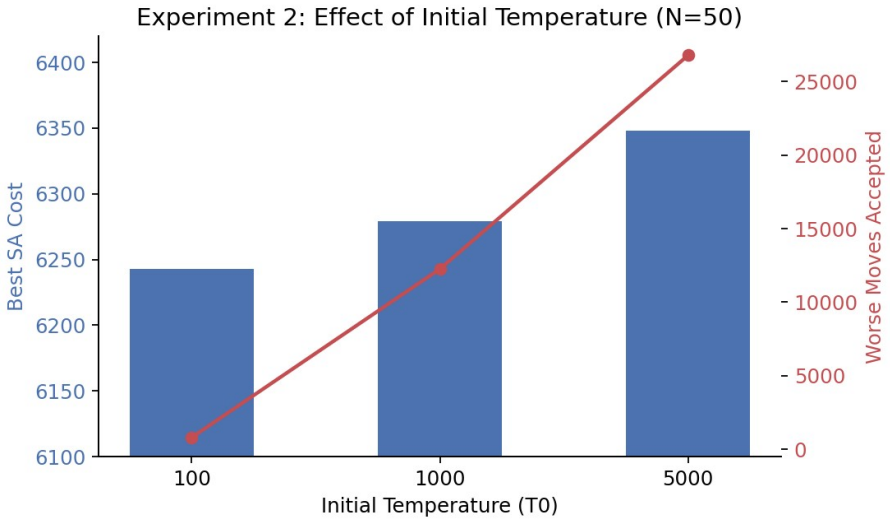


Figure 3. Best cost (bars, left axis) and worse-moves-accepted (line, right axis) vs. T0.

The higher T0 gets, the more willing SA is to accept worse moves early on — that part is expected. Going from T0=100 to T0=5000, the number of worse moves accepted jumps from 789 to almost 27,000, over 34 times more. What's a bit surprising is that the lowest temperature (T0=100) actually gave the best final result (6243.13), while the highest (T0=5000) gave the worst (6348.17).

My guess for why: with a fixed iteration budget, a very high T0 means the run spends a lot of its "budget" wandering around loosely before it ever settles down to fine-tune the solution. T0=100, on the other hand, spends almost the whole run in a more careful, exploitative mode, so it ends up polishing the tour more. Runtime barely changed between the three, since it's really the iteration count that drives time, not T0 itself. The takeaway is that a higher starting temperature isn't automatically better — it needs to match the iteration budget you actually have, otherwise you just burn iterations exploring instead of improving.

4.3 Experiment 3: Different Cooling Rates

With N=50, T0=1000 and a greedy start kept fixed, the cooling rate α was tested at 0.90, 0.95, and 0.995. A smaller α cools down faster, so the temperature drops below T_min sooner and the run ends early — even though the iteration cap is the same 100,000 for all three.

Cooling Rate (α)	Best SA Cost	Total Iterations	Accepted	Worse Accepted	SA Time (s)
0.9	6461.82	13200	1382	654	0.002187
0.95	6468.62	27000	2494	1180	0.004491
0.995	6279.45	100000	24466	12254	0.017506

Table 3. Effect of cooling rate α on Simulated Annealing (N=50).

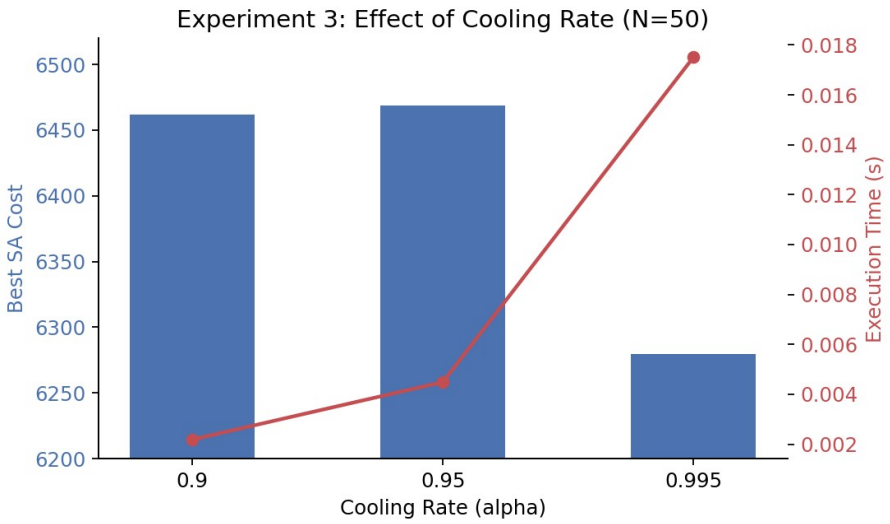


Figure 4. Best cost (bars, left axis) and execution time (line, right axis) vs. cooling rate.

Cooling fast (α=0.90) means the run only gets through about 13,200 iterations before it stops, and it shows — it gives the worst result of the three (6461.82), though it's also by far the quickest, roughly 8 times faster than the slowest setting. α=0.95 doesn't do much better (6468.62) even with roughly double the iterations, since the temperature still bottoms out before the search has really had a chance to refine anything.

Slow cooling (α=0.995) is the only one that actually uses the full 100,000 iterations, and it gives the best result by a clear margin (6279.45), about 2.8–2.9% better than the faster schedules. The cost is time — it takes 4 to 8 times as long to run. This is really the core trade-off with SA: cool slowly and you get more chances to search and refine, but it costs you runtime. If time is tight, a middle-ground α (say 0.95–0.99) combined with more moves tried per temperature step might get some of that quality back without the full runtime hit.

4.4 Experiment 4: Random vs. Greedy Initialization

With N=50 and the baseline settings (T0=1000, α=0.995) fixed, both starting methods — random and greedy — were each run 5 times with different seeds, since SA gives a different result every time it runs and one run alone wouldn't tell us much.

Seed	Random Init: SA Best	Greedy Init: SA Best
1	6078.20	6279.45
2	6202.56	6120.73
3	6304.98	6090.02
4	6201.85	6295.08
5	6239.99	6110.24

Table 4. Per-seed best SA cost for random vs. greedy initialization (N=50).

Initialization	Best	Average	Worst	Avg. SA Time (s)
Random	6078.20	6205.52	6304.98	0.017380
Greedy	6090.02	6179.10	6295.08	0.017275

Table 5. Best, average, worst, and average execution time across 5 runs, by initialization method.

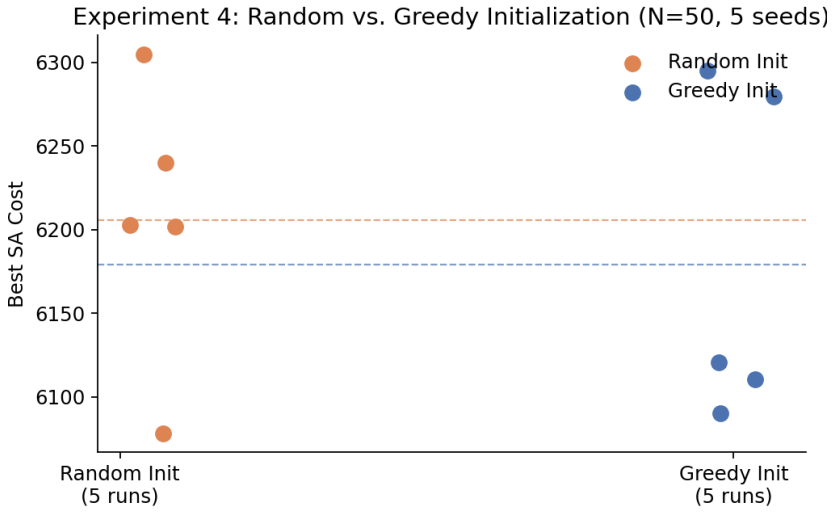


Figure 5. Spread of best SA cost across 5 seeds for each initialization method (dashed lines mark the mean).

Greedy starts off in much better shape — its starting cost is around 7000, compared to roughly 24,000–27,000 for a random tour, which is about 3.5–4 times worse. But after running the full 100,000 iterations, both end up in a pretty similar final range. Averaged across the 5 runs, greedy initialization comes out slightly ahead (6179.10 vs 6205.52, about 0.4% better) and also has a better worst-case run. Interestingly though, the single best individual run overall came from random initialization (6078.20). Runtime was basically the same either way, which makes sense — it's the iteration count, not the starting tour, that decides how long the run takes.

So the starting point matters less than you'd expect. Given enough iterations, SA's own moves and acceptance rule are strong enough to dig a random tour out of a bad spot and get it close to where greedy started. Greedy initialization gives a head start and slightly steadier results on average, but it doesn't guarantee the best outcome on any single run — random got lucky at least once here.

5. Complexity Analysis

5.1 Greedy

At every step, the algorithm has to check all the unvisited cities to find the cheapest one, which is O(N) work, and it does this N times — so overall it's $O(N^2)$. This matches what the assignment expects, and it also matches what was actually measured: greedy's

runtime went from about 2 microseconds at N=10 up to about 92 microseconds at N=200, growing roughly with the square of N.

5.2 Simulated Annealing

Let I be the number of iterations actually run (it's capped by the iteration limit, but can also stop earlier if T drops below T_min quickly). Each iteration does two things: generates a neighbor by reversing a segment of the tour, which is O(N) in the worst case, and then works out the cost of that new tour. If you recompute the whole tour's cost from scratch each time, that's another O(N), giving O(I × N) overall, which is what the assignment mentions as the straightforward approach.

There's a faster way to do it, though. Since a 2-opt move only changes two connections in the tour, you don't actually need to recompute the whole cost — you can just look at the four cities involved in the swap and work out how much the cost changed in O(1) time. The segment reversal itself is still O(N) (or cheaper with a smarter data structure), but skipping the full recomputation saves a lot of work. The measured times back this up: SA's runtime grew from about 0.0084s at N=10 to 0.0512s at N=200, which is roughly linear in N — in line with an O(I × N) implementation rather than a much slower O(I × N^2) one.

6. Making Sure the Tours Are Valid

A validateTour() function was used throughout testing to check that every tour, from either algorithm, actually met the basic requirements:

- It starts at City 0 and (once the return trip is added) ends back at City 0.
- Every city from 0 to N−1 shows up exactly once — nothing missing, nothing repeated.
- The tour has exactly N cities before the return trip.

Every tour produced across all five instances and all four experiments passed these checks, so both the greedy construction and every 2-opt move made by SA kept the tour valid the whole way through.

7. Conclusion

Overall, the results back up what the assignment sets out to show. Greedy is fast but shortsighted, and Simulated Annealing can meaningfully improve on it — by 10 to 23% for instances up to 100 cities — by using 2-opt moves and occasionally accepting worse solutions to get past the dead ends greedy gets stuck in.

That said, SA's advantage isn't guaranteed. At N=200, with the same fixed iteration budget, it didn't beat greedy at all, which shows that the settings (iterations, cooling rate) really need to scale up as the problem gets bigger. Out of everything tested, the cooling rate made the biggest difference to solution quality — cooling too fast wastes the whole run. Initial temperature and starting tour mattered too, but less dramatically: too high a T0 wastes iterations on exploring instead of refining, and a greedy starting tour

gives slightly better and more consistent results than a random one, though it doesn't guarantee the single best run.

In short: use greedy when you just need something quick and workable, and use SA when solution quality actually matters and you can afford to tune the iteration budget and cooling schedule properly.